

April 13, 2026

QUADCOPTER ARM OPEN AND CLOSED LOOP CONTROL

MMAE 415 – Aerospace Laboratory II

Abstract

The goal of this lab experiment was to set up and calibrate the closed loop angle control of a single quadcopter arm using a brushless DC motor controlled by a pulse width modulation (PWM) signal, and compare it to open loop control. The quadcopter arm was commanded to move from a pitch angle of -70 degrees to -55 degrees in a step input, and an accelerometer attached to the quadcopter arm provided data that was used to calculate pitch angle. This angle was put into a PID controller which gave angle feedback and could be used to adjust the duty cycle controlling the motor throttle. The controller was implemented in Simulink and tuned manually in dSpace ControlDesk, with each term (P, I, D) being tuned in sequence to achieve stability after the step input. The results showed that while open loop control had a steady state error, implementing a proportional (P), proportional integral (PI), and proportional integral derivative (PID) controllers improved the response time, steady state error, and reduced oscillations. While the implementation of a PID controller improved the systems response, more work is needed to tune the controller to achieve greater angle accuracy.

Kaleb Martiny

ILLINOIS INSTITUTE OF TECHNOLOGY | Chicago, Illinois

Table of Contents

Introduction.....	2
Relevant Equations	3
Experimental Setup.....	4
Results.....	6
Discussion of Results	8
Conclusion	9
Appendix A	10
Appendix B	14
References	16

Introduction

In order for a quadcopter to fly, precise adjustments need to be made to the input of the motors to be able to generate the necessary thrust for flight. In the previous experiment, open loop control was explored with the creation of a static calibration map, which could be used to estimate the necessary duty cycle based on an input pitch angle. While the open loop system was able to respond to a commanded pitch angle input, the accuracy of the system could not be improved due to its open loop nature. In an open loop system, an input is created or commanded, and as it is run through a plant or controller, generates some output. In this case, the desired quadcopter arm angle was input, and using the inverted linear regression from the static map which serves as the controller, a duty cycle was output and sent to the motor. This works well until error is present, as an open loop system has no way to account and correct the errors. For this type of system, closed loop control is better suited. Closed loop control takes an input, runs it through the plant or controller and generates an output, just like open loop control. However, this output is then fed back into the input and compared, which generates an error. This error allows the system to correct itself, ideally moving toward the original input as the loop is iterated on (Franklin, Powell, & Emami-Naeini, 2020).

The controller used in this experiment was a proportional, integral, and derivative (PID) controller that was manually tuned. A PID controller utilizes the closed loop feedback error system to make the system reach and maintain a desired input. The proportional term responds to the immediate amount of error and provides a correction that is proportional to that error. A larger error will result in a larger corrective response. While the proportional term is able to provide large corrections, it is not able to eliminate steady state error, and too large of a proportional gain can cause oscillations. The integral term responds to the accumulated error that builds over time. It is mainly used to reduce the amount of steady state errors in the system. Too large of an integral can cause the system to slow down, increase the amount of overshoot experienced, and cause oscillations. The derivative term responds to how fast the error is accumulating. It helps to reduce overshoot and stabilize the system. Too large a derivative can add unwanted noise to the signal. Using all three together lets the response of a system be fully controlled depending on the desired output (Franklin S. A., 2018).

The main objective of this experiment was to evaluate the time response of a quadcopter arm moving from a commanded angle of -70 degrees to -55 degrees and compare the systems reaction under four different conditions: open loop control, proportional (P) control, proportional integral (PI) control, and proportional integral derivative (PID) control. The response of the system under these conditions is then compared with each other to determine if the values found improved the system's response, and what further work could be done to improve the system's accuracy.

Relevant Equations

To calculate the current angle of the quadcopter arm, a sensitivity constant and offset constant were needed and can be calculated using:

$$V_x = V_{x,offset} + S_x \left(\frac{a_x}{g} \right) \text{ and } V_z = V_{z,offset} + S_z \left(\frac{a_z}{g} \right) \quad (1)$$

Where:

$V_x, V_z = \text{Output Voltage}$

$V_{x,offset}, V_{z,offset} = \text{Voltage When the Directional Component and Gravity Equal}$

$S_x, S_z = X \text{ and } Z \text{ Axes Sensitivities}$

$a_x, a_z = \text{Acceleration in } X \text{ and } Z$

$g = \text{Gravitational Acceleration}$

In order to determine the sensitivity constants, the quadcopter arm was held at 0° and -90° . The attached accelerometer was connected to an oscilloscope so the output voltages could be read and recorded. At -90° vertical, the Z axis voltage, $V_{z,offset}$, is at its minimum because the acceleration in the Z axis direction is 0. The X axis voltage, V_x , is at its maximum as the acceleration in the X axis direction is 1g. At 0° horizontal, the X axis voltage, $V_{x,offset}$, is at its minimum because the acceleration in the X axis direction is 0. The Z axis voltage, V_z , is at its maximum because the acceleration in the Z axis direction is 1g. With voltages for Y and Z recorded at 0° and -90° , Equation 1 can be used to calculate the required sensitivities.

With the sensitivities and voltages, the angle of the quadcopter arm can then be computed using the following trigonometric relations:

$$\theta = -\sin^{-1} \left(\frac{V_x - V_{x,offset}}{S_x} \right) \text{ and } \theta = -\cos^{-1} \left(\frac{V_z - V_{z,offset}}{S_z} \right) \quad (2)$$

Because of the small values of sin and cos near 0° and -90° , a weighted average is computed using the output angles from Equation 2 using:

$$\theta_{Avg} = \theta_x \cos^2(\theta_x) + \theta_z \sin^2(\theta_x) \quad (3)$$

To reduce noise, a first order low pass filter was applied to incoming signal using a transfer function with a cutoff frequency of 5 Hz. The transfer function time constant was calculated using:

$$f_c = \frac{1}{2\pi\tau} \quad (4)$$

Where:

$f_c = \text{Cutoff Frequency}$

$\tau = \text{Time Constant}$

The resulting time constant from Equation 4, calculated as $\tau = 0.03183$, was placed into the following transfer function:

$$\frac{1}{\tau s + 1} \quad (5)$$

Experimental Setup

This experiment was set up using a single quadcopter arm attached to a pivot at one end to allow it to rotate from $\sim -90^\circ$ to $\sim 90^\circ$ freely. The other end of the arm had the brushless DC motor, propeller, accelerometer, and electronic speed controller (ESC). The accelerometer was powered with 3.3 VDC, and its output voltages were measured using the analog to digital converter (ADC) on a MicroLabBox DS1202. Channels 1 and 2 were used for the X and Z axes. The other equipment used in this experiment were MATLAB, Simulink, dSpace ControlDesk, a Siglent SDS 1102CML+ oscilloscope, a BK Precision 1760A power supply, a Global Industries PB-505 bread board, and an external battery. During initial setup, the propeller and battery were removed for safety reasons.

A Simulink model was created to convert the voltage outputs from the accelerometer into a pitch angle. The low pass filter from Equation 5 was used, along with Equations 2 and 3 to calculate an average pitch angle. A manual PID controller was also implemented so that P, I, and D values could be manually adjusted in ControlDesk for tuning purposes. A feedforward control static map was put in parallel with the PID controller. The difference between the command angle input and the accelerometer average angle output was used as the error signal, which was fed directly into the PID setup. The initial baseline controller parameters were set to $K_P = 0.0001$, $K_I = 0$, and $K_D = 0$. The integrator included a manual switch to reset the accumulated error. The derivative included a first order low pass filter with $\tau = 0.159$. Once the Simulink model was created, it was built and exported to ControlDesk, where real time plots were set up to display the angle and duty cycle, and numeric inputs were added for each PID term, as well as the integrator reset switch.

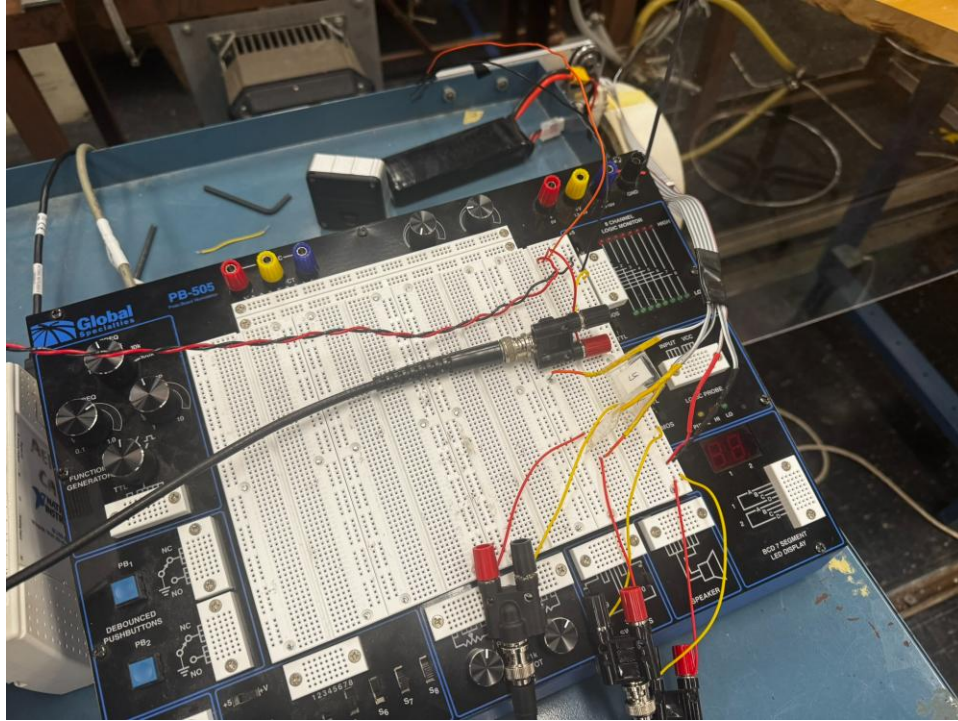


Figure 1: Circuit setup for PWM control and accelerometer readings. The accelerometer data and power are connected to the bottom left of the breadboard. The accelerometer is powered by the BK Precision 1760A power supply with 3.3 VDC, and the data is read by the MicroLabBox ADC on channels 1 and 2. The electronic speed controller (ESC) is connected to the breadboard on the top right. The output from the MicroLabBox is connected to the ESC and is read by the oscilloscope for signal verification.

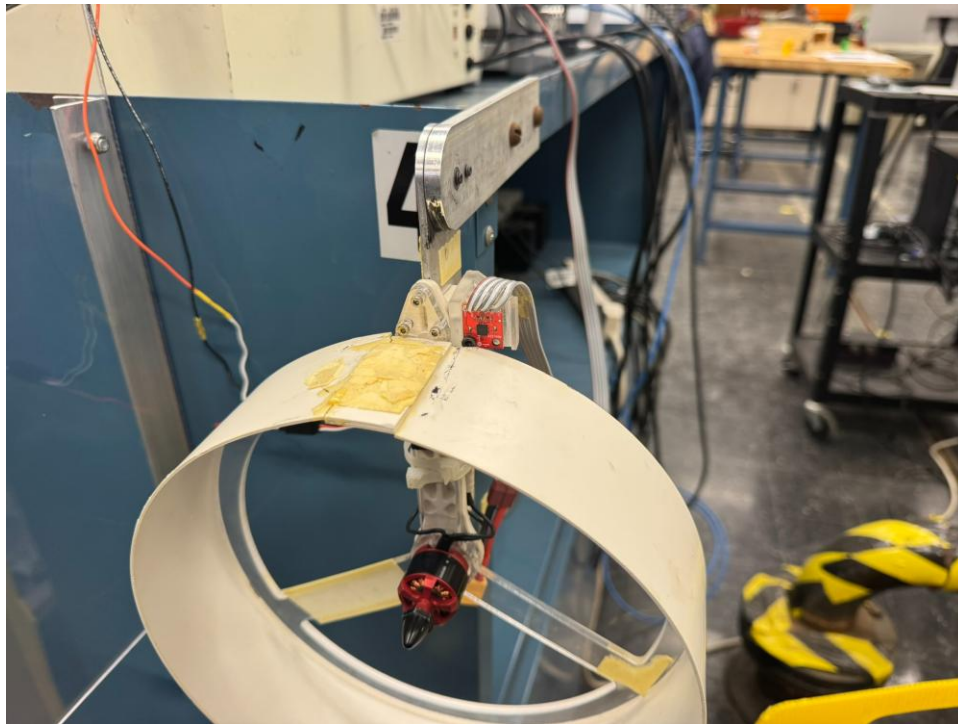


Figure 2: Single quadcopter arm with DC motor and accelerometer. The propeller is not attached yet for safety during setup. The accelerometer is connected to the breadboard for power and data handling. The DC motor is connected to an ESC and battery. The ESC is connected to the breadboard and uses the output duty cycle to control the DC motor.

Results

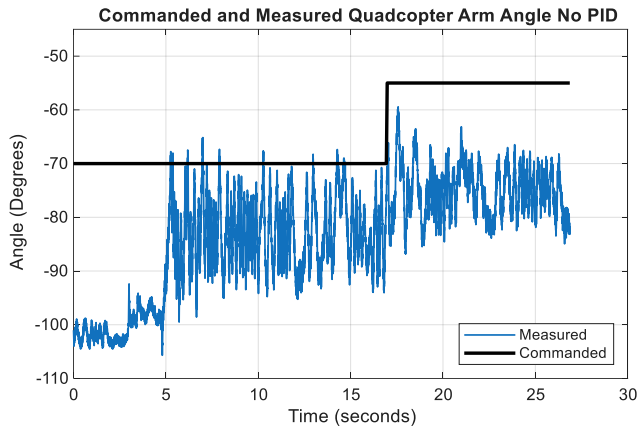


Figure 3: Recorded data for the commanded step input from -70° to -55° . A $\sim 10^\circ$ offset can be seen before the step input, while a $\sim 20^\circ$ offset can be seen after the input. The data before 5 seconds can be ignored as the motor had not started spinning yet.



Figure 4: Recorded data for the commanded step input from -70° to -55° with a proportional controller. The value used for $K_p = 4E - 5$. A 13° offset is present before the step input. A 20° offset is seen after the step input.

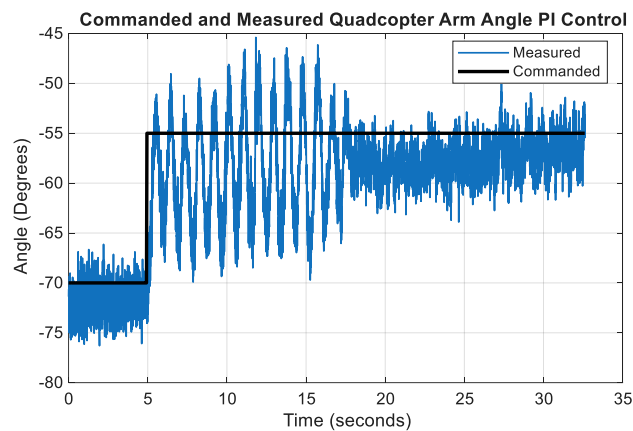


Figure 5: Recorded data for the commanded step input from -70° to -55° with a proportional integral controller. The value used for $K_i = 5E - 5$. A 5° offset is present before the step input. A steady state offset of 3° is present after the step input.

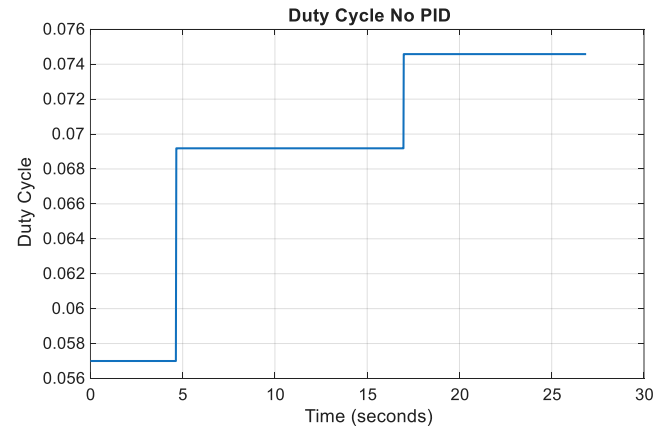


Figure 6: Duty cycle input for open loop control with a step input from -70° to -55° . Before 5 seconds can be ignored.

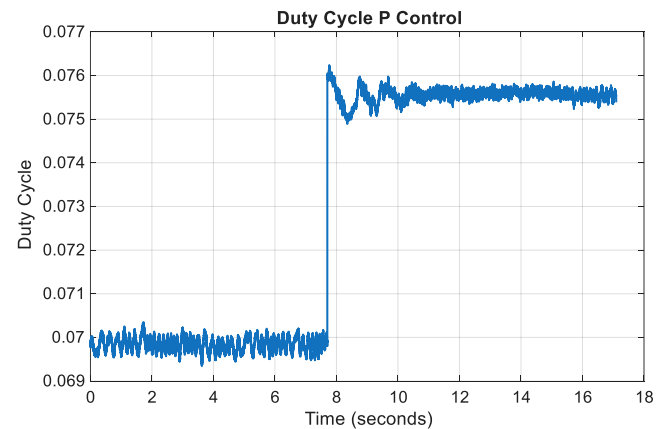


Figure 7: Duty cycle input for proportional control with a step input from -70° to -55° . Oscillations occur after the step and are mostly damped out in steady state.

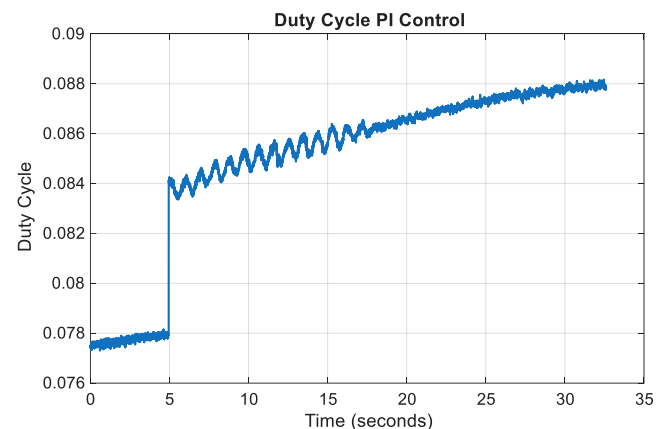


Figure 8: Duty cycle input for proportional integral control with step input from -70° to -55° . The duty cycle input climbs as error accumulates and the system continues to try to correct itself.

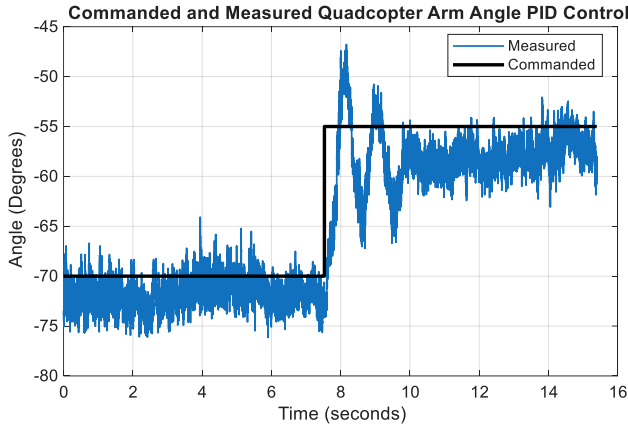


Figure 9: Recorded data for the commanded step input from -70° to -55° with a proportional integral derivative controller. The value used for $K_D = 5E - 6$. A 3° offset is present before the step input. A rough steady state offset of 3° is present after the step input, however the system should have been run longer for better steady state response.

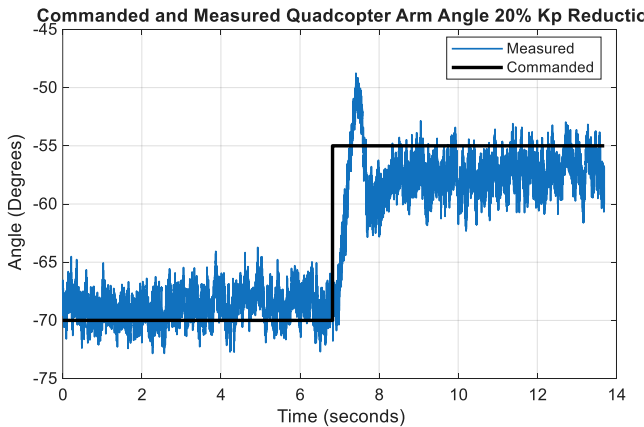


Figure 10: Recorded data for the commanded step input from -70° to -55° with a proportional integral derivative controller. The proportional gain K_P was reduced by 20% to help improve the system response. A 2° offset is present before the step input. A 3° offset is present after the step input.

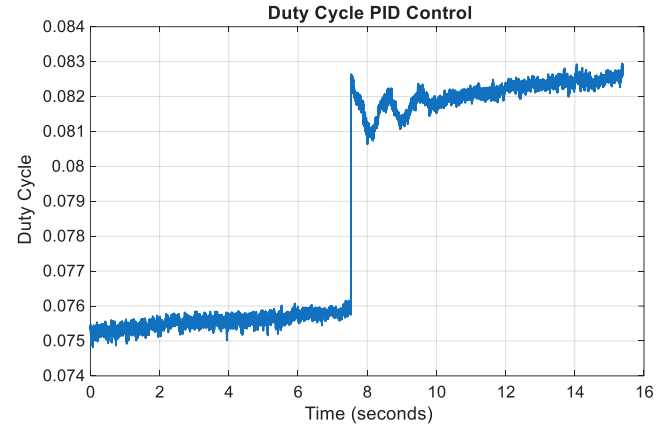


Figure 11: Duty cycle input for proportional integral derivative control with step input from -70° to -55° . Much fewer oscillations are present after the step input.

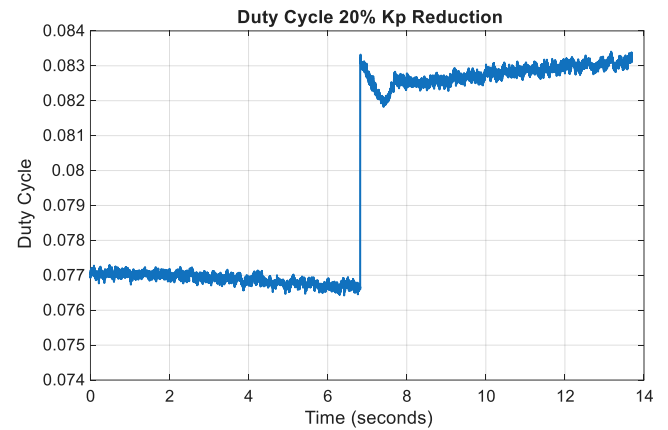


Figure 12: Duty cycle input for open loop control with step input from -70° to -55° . Even fewer oscillations are present after the input with a more controlled steady state response.

Figures 3-5, 9, and 10 show the quadcopter arm's response to the initial -70° command, as well as the step input to -55° . Figure 3 records the time response of the system with open loop feedback, where there is a consistent offset from the commanded input that does not improve with time. Figure 4 records the time response with a proportional (P) controller with $K_P = 4E - 5$. There is still an offset present before and after the step input, and the system reacts more distinctively to the step input, but the steady state response is not improved. Figure 5 shows the time response with a proportional integral (PI) controller with $K_I = 5E - 5$. The offset seen in Figures 3 and 4 is greatly reduced, and the tracking with the step input tracks much closer, however large oscillations are present for about 12 seconds until the system begins to stabilize. Figure 9 shows the time response with a proportional integral derivative (PID) controller with $K_D = 5E - 6$. Again, the offset seen in previous models has been greatly reduced, and the large oscillations after the step input are more controlled, only lasting about 3 seconds. Figure 10 shows the time response of the PID controller with K_P reduced by 20%. An offset of about 3° still remains before and after the step input however the signal is much more stable than previous models. There is only one large oscillation after the step input, which is damped out in about 1 second, with the system becoming stable immediately afterwards.

Figures 6-8, 11, and 12 show the time response of the duty cycle used to control the motor. Figure 6 shows the open loop duty cycle, which is static until the step input. Figure 7 shows the P controller duty cycle, where signal oscillations are now seen as the system feeds back on itself and attempts corrections. Figure 8 shows the PI controller duty cycle. More significant corrections are seen with the increasing duty cycle. Figure 11 shows the PID controller duty cycle, where the large oscillations after the step input are reduced compared to Figure 8. Figure 12 shows the 20% reduction in K_p duty cycle, where the large oscillations are further damped out.

A MATLAB script was used to generate Figures 3-12. The data used was exported from ControlDesk as a .mat file and imported into MATLAB for processing and analysis. The full script used can be found in Appendix A. The Simulink model used for data collection and PID controller implementation can be found in Appendix B.

Discussion of Results

The results of this experiment show the differences between open loop and closed loop control, specifically how proportional, integral, and derivative terms can be used to change the time response of a quadcopter arm. The open loop control showed significant steady state offsets of about -10° before the step and -20° after. Because an open loop system relies solely on the inverted static calibration map to estimate the required duty cycle, it has no way to account for or correct errors in the system output. Adding the proportional controller with $K_p = 4E - 5$ generated a corrective response using the error between the commanded and measured angle. While the P controller allowed the system to react to the step input more quickly and distinctively, it resulted in a large 20° steady state offset and initial oscillations before being mostly damped out. A larger K_p gain generally increases the response rate, but it was limited in this experiment because higher proportional gains led to severe system instability.

To help eliminate some of the steady state error left by the proportional controller, an integral gain of $K_I = 5E - 5$ was added. The integral term successfully responded to the accumulated error over time, which is shown in Figure 8 where the duty cycle climbs as the system continuously attempts to correct itself. This effectively reduced the steady state error to just 3° . However, compared to the open loop configuration, the PI controller introduced more sustained, high amplitude oscillations right after the step input that were not corrected for several seconds. Adding the derivative term $K_D = 5E - 6$ to form a full PID controller mostly addressed these oscillations. By responding to the rate at which the error was changing, the derivative term stabilized the system and significantly decreased the large oscillations observed during the step change, all while maintaining the reduced 3° steady state offset.

To further optimize the quadcopter arms angle determination, the proportional gain was decreased by 20% from its initial value. This tuned PID configuration showed even fewer oscillations and had a much more controlled steady state response. Despite these improvements,

offsets were still seen, with a roughly 3° offset present both before and after the step input. These errors likely stem from experimental factors, including mechanical friction at the arm's pivot, small variations in the battery's power delivery, or sensor noise from the accelerometer. It should be noted that during setup and testing 3 different batteries were needed as they kept running out of power. Additionally, as noted in the results, the system should have been recorded for longer to allow the quadcopter arm to achieve a true steady state response, which would allow for more accurate controller tuning in future iterations. Even with full PID control, overshoot was still present by about 5° , meaning the PID values could use further tuning.

Conclusion

This experiment was able to successfully analyze the time response of a quadcopter arm fitted with a DC motor and propeller undergoing a commanded step input from -70° to -55° under both open and closed loop control. The open loop control was based only on a static calibration map and was unable to correct system errors which resulted in large offsets from the commanded angles. Adding a closed loop feedback controller allowed the system to correct itself to a certain extent. The proportional gain added allowed the system to have more immediate correction to the step input but did not help improve the steady state response. The integral gain helped to reduce the steady state error but introduced large oscillations after the step input that took time to be damped out. The derivative gain helped to dampen the large oscillations and improved the steady state response but was unable to reduce system overshoot.

One major weakness of this experiment was that the system was not allowed to run for long enough to fully capture its steady state response. Figures 5, 9, and 10 show this, where the average measured angle appears to be trending upward toward the commanded angle. If the system had been left to run for longer, some of the offset errors seen may have been eliminated. Future runs of this experiment should extend the data recording time, perhaps as far as a minute to fully capture the systems response.

Appendix A

```

clc; clear;

data_no = load("no_pid.mat");
data_P = load("P_control.mat");
data_PI = load("PI_control.mat");
data_PID = load("PID_control.mat");
data_16 = load("step16.mat");

time_no = data_no.no_pid.X(1).Data;
meas_angle_no = data_no.no_pid.Y(1).Data;
duty_no = data_no.no_pid.Y(2).Data;
comm_angle_no = data_no.no_pid.Y(3).Data;

time_P = data_P.P_control.X(1).Data;
meas_angle_P = data_P.P_control.Y(1).Data;
duty_P = data_P.P_control.Y(2).Data;
comm_angle_P = data_P.P_control.Y(3).Data;

time_PI = data_PI.PI_control.X(1).Data;
meas_angle_PI = data_PI.PI_control.Y(1).Data;
duty_PI = data_PI.PI_control.Y(2).Data;
comm_angle_PI = data_PI.PI_control.Y(3).Data;

time_PID = data_PID.PID_control.X(1).Data;
meas_angle_PID = data_PID.PID_control.Y(1).Data;
duty_PID = data_PID.PID_control.Y(2).Data;
comm_angle_PID = data_PID.PID_control.Y(3).Data;

time_16 = data_16.step16.X(1).Data;
meas_angle_16 = data_16.step16.Y(1).Data;
duty_16 = data_16.step16.Y(2).Data;
comm_angle_16 = data_16.step16.Y(3).Data;
% No PID plots (figure 1)
figure(1);
plot(time_no, meas_angle_no, 'LineWidth', 1)
hold on
grid on
h_comm = plot(time_no, comm_angle_no, 'k', 'LineWidth', 2);
uistack(h_comm, 'top');
title('Commanded and Measured Quadcopter Arm Angle No PID')
xlabel('Time (seconds)')
ylabel('Angle (Degrees)')
legend('Measured', 'Commanded', 'Location', 'southeast')

```

```
ylim([-110 -45])
hold off

% No PID duty (figure 2)
figure(2);
plot(time_no, duty_no, 'LineWidth', 1.2)
grid on
title('Duty Cycle No PID')
xlabel('Time (seconds)')
ylabel('Duty Cycle')

% P control plots (figure 3)
figure(3);
plot(time_P, meas_angle_P, 'LineWidth', 1)
hold on
grid on
h_comm = plot(time_P, comm_angle_P, 'k', 'LineWidth', 2);
uistack(h_comm, 'top');
title('Commanded and Measured Quadcopter Arm Angle P Control')
xlabel('Time (seconds)')
ylabel('Angle (Degrees)')
legend('Measured', 'Commanded', 'Location', 'best')
ylim([-95 -45])
hold off

% P control duty (figure 4)
figure(4);
plot(time_P, duty_P, 'LineWidth', 1.2)
grid on
title('Duty Cycle P Control')
xlabel('Time (seconds)')
ylabel('Duty Cycle')

% PI control plots (figure 5)
figure(5);
plot(time_PI, meas_angle_PI, 'LineWidth', 1)
hold on
grid on
h_comm = plot(time_PI, comm_angle_PI, 'k', 'LineWidth', 2);
uistack(h_comm, 'top');
title('Commanded and Measured Quadcopter Arm Angle PI Control')
xlabel('Time (seconds)')
ylabel('Angle (Degrees)')
legend('Measured', 'Commanded', 'Location', 'best')
hold off
```

```

% PI control duty (figure 6)
figure(6);
plot(time_PI, duty_PI, 'LineWidth', 1.2)
grid on
title('Duty Cycle PI Control')
xlabel('Time (seconds)')
ylabel('Duty Cycle')

% PID control plots (figure 7)
figure(7);
plot(time_PID, meas_angle_PID, 'LineWidth', 1)
hold on
grid on
h_comm = plot(time_PID, comm_angle_PID, 'k', 'LineWidth', 2);
uistack(h_comm, 'top');
title('Commanded and Measured Quadcopter Arm Angle PID Control')
xlabel('Time (seconds)')
ylabel('Angle (Degrees)')
legend('Measured', 'Commanded', 'Location', 'best')
hold off

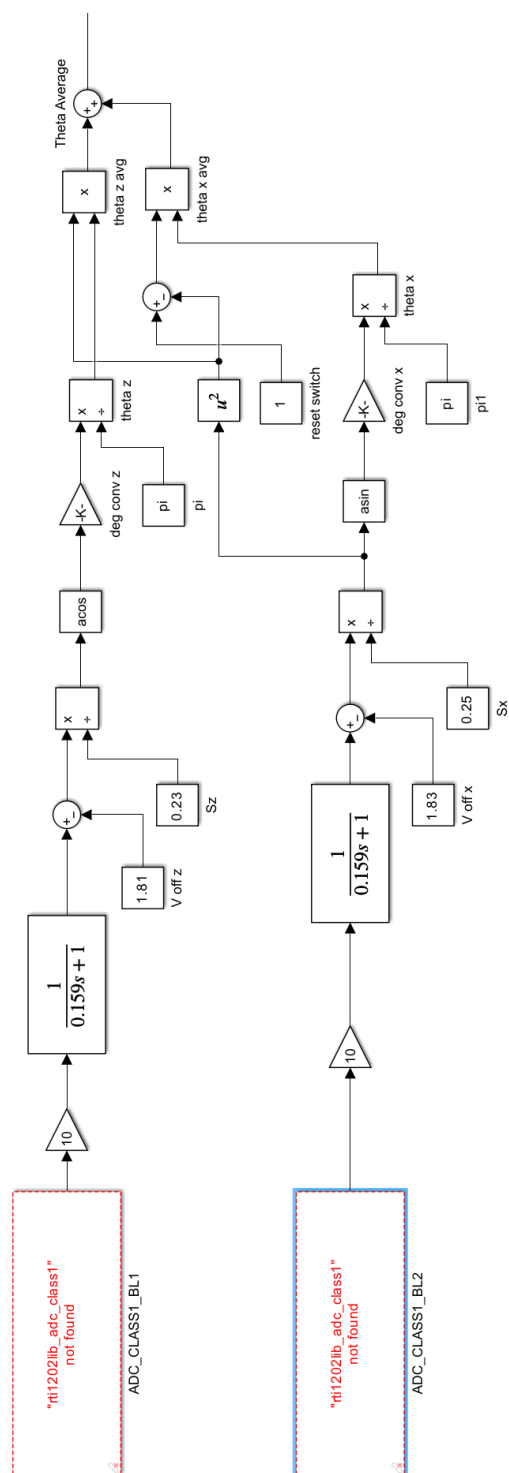
% PID control duty (figure 8)
figure(8);
plot(time_PID, duty_PID, 'LineWidth', 1.2)
grid on
title('Duty Cycle PID Control')
xlabel('Time (seconds)')
ylabel('Duty Cycle')
ylim([0.074 0.084])

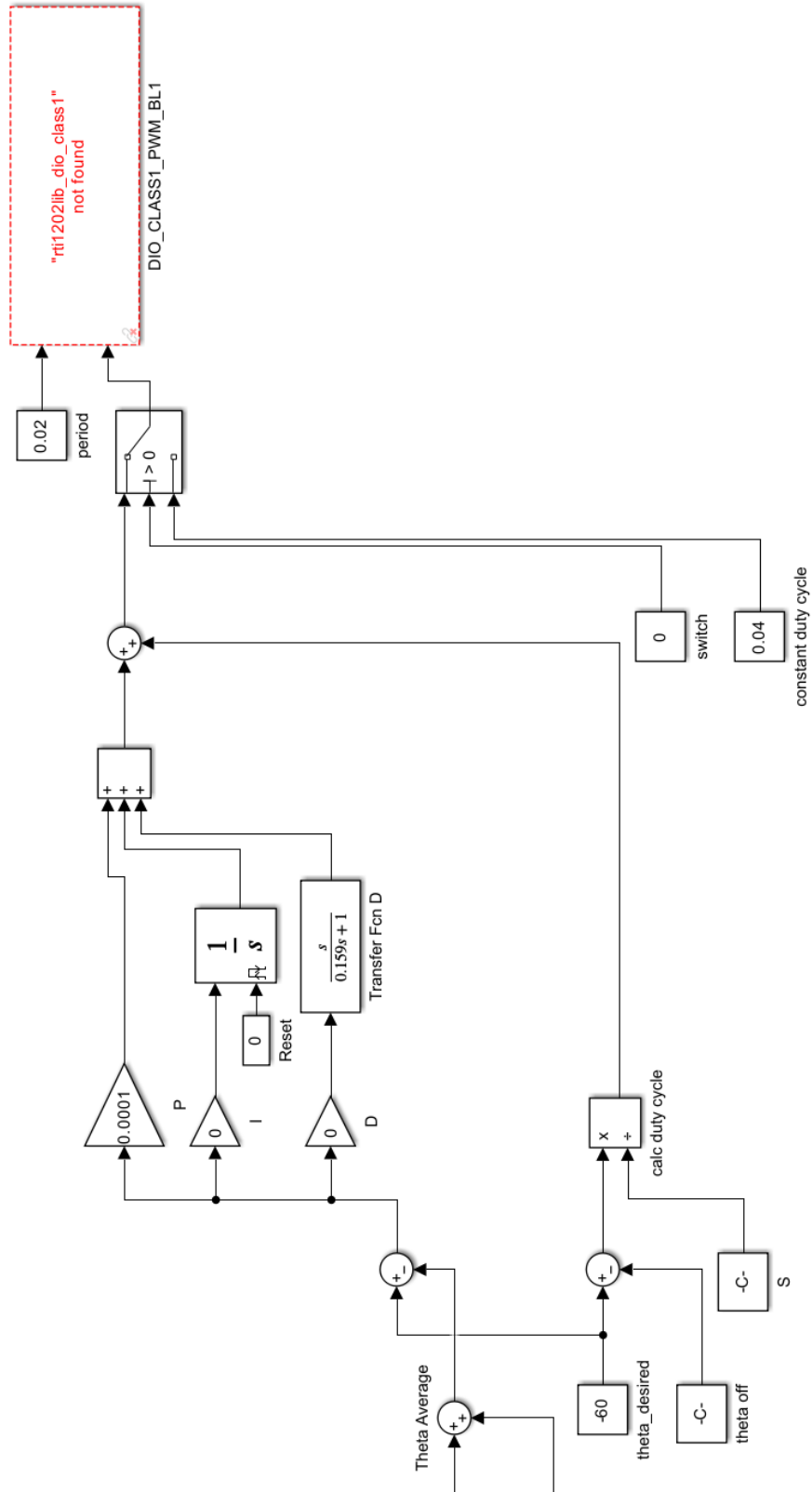
% 0.2kp control plots (figure 9)
figure(9);
plot(time_16, meas_angle_16, 'LineWidth', 1)
hold on
grid on
h_comm = plot(time_16, comm_angle_16, 'k', 'LineWidth', 2);
uistack(h_comm, 'top');
title('Commanded and Measured Quadcopter Arm Angle 20% Kp Reduction')
xlabel('Time (seconds)')
ylabel('Angle (Degrees)')
legend('Measured', 'Commanded', 'Location', 'best')
hold off

% 0.2kp control duty (figure 10)
figure(10);
plot(time_16, duty_16, 'LineWidth', 1.2)

```

```
grid on
title('Duty Cycle 20% Kp Reduction')
xlabel('Time (seconds)')
ylabel('Duty Cycle')
ylim([0.074 0.084])
```





References

- Franklin, G. F., Powell, D. J., & Emami-Naeini, A. (2020). *Feedback Control of Dynamic Systems*. New York: Pearson.
- Franklin, S. A. (2018). *Control Theory Tutoria: Basic Concepts Illustrated by Software Examples*. Irvine: Springer Open.